

PentestAgent: Incorporating LLM Agents to Automated Penetration Testing

Xiangmin Shen*
Northwestern University
Evanston, Illinois, USA
xiangminshen2019@u.northwestern.edu

Lingzhi Wang*
Northwestern University
Evanston, Illinois, USA
lingzhiwang2025@u.northwestern.edu

Zhenyuan Li
Zhejiang University
Hangzhou, Zhejiang, China
lizhenyuan@zju.edu.cn

Yan Chen
Northwestern University
Evanston, Illinois, USA
ychen@northwestern.edu

Wencheng Zhao
Ant Group
Hangzhou, Zhejiang, China
wencheng.zwc@antgroup.com

Dawei Sun
Ant Group
Hangzhou, Zhejiang, China
david.sdw@antgroup.com

Jiashui Wang
Zhejiang University
Hangzhou, Zhejiang, China
12221251@zju.edu.cn

Wei Ruan
Zhejiang University
Hangzhou, Zhejiang, China
ruanwei@zju.edu.cn

Abstract

Penetration testing is a critical technique for identifying security vulnerabilities, traditionally performed manually by skilled security specialists. This complex process involves gathering information about the target system, identifying entry points, exploiting the system, and reporting findings. Despite its effectiveness, manual penetration testing is time-consuming and expensive, often requiring significant expertise and resources that many organizations cannot afford. While automated penetration testing methods have been proposed, they often fall short in real-world applications due to limitations in flexibility, adaptability, and implementation.

Recent advancements in large language models (LLMs) offer new opportunities for enhancing penetration testing through increased intelligence and automation. However, current LLM-based approaches still face significant challenges, including limited penetration testing knowledge and a lack of comprehensive automation capabilities. To address these gaps, we propose PENTESTAGENT, a novel LLM-based automated penetration testing framework that leverages the power of LLMs and various LLM-based techniques like retrieval augmented generation (RAG) to enhance penetration testing knowledge and automate various tasks. Our framework leverages multi-agent collaboration to automate intelligence gathering, vulnerability analysis, and exploitation stages, reducing manual intervention. We evaluate PENTESTAGENT using a comprehensive benchmark, demonstrating superior performance in task completion and overall efficiency.

*Both authors contributed equally to this work.

CCS Concepts

• Security and privacy → Penetration testing; • Computing methodologies → Multi-agent systems.

Keywords

Penetration Testing, Large Language Model, Agent

ACM Reference Format:

Xiangmin Shen, Lingzhi Wang, Zhenyuan Li, Yan Chen, Wencheng Zhao, Dawei Sun, Jiashui Wang, and Wei Ruan. 2025. PentestAgent: Incorporating LLM Agents to Automated Penetration Testing. In *ACM Asia Conference on Computer and Communications Security (ASIA CCS '25)*, August 25–29, 2025, Hanoi, Vietnam. ACM, New York, NY, USA, 17 pages. <https://doi.org/10.1145/3708821.3733882>

1 Introduction

Penetration testing is a widely adopted technique for proactively identifying security vulnerabilities. This process involves gathering information about the target system (reconnaissance), identifying possible entry points, attempting to exploit the system, and reporting the findings. [14] Traditionally, penetration testing has been a complex manual process requiring highly skilled security specialists with extensive experience. Testers typically write their own exploits, master public domain tools, and perform numerous tedious and time-consuming tasks. [43] According to Rapid7's Under the Hoodie report, penetration testing takes an average of 80 hours, with significant outliers taking several hundred hours. [7] Consequently, manual penetration testing often necessitates large, diverse teams, which most organizations cannot afford.

Although automated penetration testing has been a concept for over a decade, a significant gap remains between the proposed methods and their real-world application. Early works [5, 30, 36] primarily modeled attack planning as an attack graph problem [3] in a deterministic and fully observable world. However, such an approach imposes limitations: it assumes complete observability from the defenders' standpoint and lacks the flexibility and adaptability



This work is licensed under a Creative Commons Attribution 4.0 International License. *ASIA CCS '25, Hanoi, Vietnam*
© 2025 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-1410-8/2025/08
<https://doi.org/10.1145/3708821.3733882>

required for dynamic environments. Later efforts [6, 15, 19, 20, 37–39, 49] addressed these shortcomings by introducing uncertainty into planning methodologies, treating attack planning as a Markov Decision Process (MDP), which model the world as states and actions as transitions between states, with a reward function encoding the “reward” for moving from one state to another. As extensions to MDP-based approaches, the subsequent works employ partially observable Markov decision process (POMDP) [37, 38] and reinforcement learning algorithms [6, 19] to account for further uncertainty in the environment and action outcomes. These advancements better align with real-world conditions where attackers possess limited knowledge of the target systems. Nevertheless, these probabilistic models focus on establishing a theoretical model for automated pentesting planning and lack the implementation aspect.

Large language models (LLMs) are rapidly evolving, showcasing impressive capabilities in a wide range of tasks, including text summarizing, data analysis, and question-answering. The powerful LLMs have gained significant attention in security applications, leading to a shift towards LLM-based security solutions that offer enhanced intelligence and automation capabilities compared to existing methods, making it possible to address the implementation gap in automated penetration testing.

Recent attempts to utilize LLMs for automating penetration testing [12, 18, 48] have shown some promising initial results. However, two crucial gaps still need to be addressed for practical use:

- 1) **Limited pentesting knowledge:** These methods heavily rely on pre-trained language models for generating actionable items. However, the training datasets for these models often lack comprehensive coverage of penetration testing techniques. This results in a limited state space and an outdated action space, reducing the effectiveness and relevance of the generated actions.
- 2) **Insufficient Automation:** Existing approaches lack the automated capabilities to adapt to various environments, including validating and debugging the suggested procedures and dynamically acquiring and applying new pentesting techniques.

Table 1: Comparison of LLM-based pentesting systems

System	State&Action Space	Online Search Augmentation	Validation& Debugging Capability
PENTESTAGENT	Large	Auto	Auto
AUTOATTACKER [48]	Unknown ¹	Manual	Manual
PENTESTGPT [12]	Unknown ¹	Manual	Manual
Happe et al. [18]	Small	No	No

¹ AUTOATTACKER and PENTESTGPT solely rely on LLMs to provide reconnaissance and attack techniques, which can be limited and outdated.

To overcome these challenges, we propose a novel LLM-based automated penetration testing framework PENTESTAGENT. Our framework aims to enhance penetration testing knowledge by continuously integrating new techniques and updating the framework’s knowledge base with the assistance of LLMs. Additionally, PENTESTAGENT establishes a robust automated penetration testing pipeline utilizing LLM techniques, incorporating validation and debugging mechanisms to ensure the effectiveness and relevance of generated actions in specific target environments. By bridging these gaps, we aim to significantly improve the practical applicability and reliability of automated penetration testing frameworks.

PENTESTAGENT employs a multi-agent design where each agent is responsible for a specific task in the penetration testing process. This flexible architecture enables the customization of toolsets across different tasks, enhancing the system’s adaptability. In addition to LLM agents, PENTESTAGENT integrates Retrieval Augmented Generation (RAG) [21] to leverage supplementary data during response synthesis and manage the context efficiently. This combination enhances the quality of the generated outputs and further reduces the need for manual intervention.

Table 1 provides a comparative overview of existing automated pentesting systems, illustrating that while frameworks like AutoAttacker and PENTESTGPT rely on manual or limited approaches, PENTESTAGENT offers a fully automated solution with a larger state and action space and advanced online search augmentation and debugging capabilities.

Moreover, our contributions extend beyond the framework itself. We introduce a comprehensive penetration testing benchmark based on VulHub’s collection of vulnerable Docker environments and Capture The Flag (CTF) challenges on HackTheBox. This benchmark spans various difficulty levels and encompasses a wide range of vulnerabilities, addressing a critical gap in current research by providing a practical and accessible evaluation framework.

To sum up, we make the following contributions:

- We design PENTESTAGENT, a LLM-based automated pentesting system that operates with minimal human intervention. PENTESTAGENT integrates multi-agent design and Retrieval Augmented Generation (RAG) techniques to enhance penetration testing knowledge and automate various tasks.
- We design a comprehensive penetration testing benchmark based on the leading open-source collection of pre-built vulnerable docker environments VulHub. This benchmark spans various levels of difficulty and encompasses a wide range of common weaknesses and vulnerabilities, providing a comprehensive and practical framework for evaluating penetration testing tools.
- We design experiments and metrics to evaluate PENTESTAGENT on our benchmark. The results demonstrate PENTESTAGENT’s superior performance in automatically completing the entire penetration testing process, as well as in individual penetration tasks.

We make our benchmark datasets and framework publicly available to facilitate further research in automated penetration testing.¹

2 Background and Related Work

2.1 Penetration Testing

Penetration testing (pentesting) is a structured, multi-stage process designed to identify security vulnerabilities in systems. According to the Penetration Testing Execution Standard (PTES) [42], pentesting consists of three main stages: intelligence gathering, vulnerability analysis, and exploitation. Penetration testing is broadly divided into external and internal assessments [7]. External assessments focus on assets exposed on internet, such as web applications, online services, and external networks, using techniques like social engineering, red teaming, and, in particular, web penetration testing.

¹ <https://github.com/nbshenxm/pentest-agent>

In contrast, internal assessments target the organization’s internal network, source code, or physical devices, typically involving code reviews and internal network compromises.

While several recent works have enhanced internal assessments using LLM-based frameworks (e.g., ChatAFL [26], FuzzGPT [13], LLift [23], and LATTE [25]), external assessments, especially web penetration testing, remain underexplored. According to Rapid7’s latest report [8], external network compromise constitutes over 80% of penetration testing tasks. This paper addresses this gap by demonstrating how PENTESTAGENT automates web penetration testing, thereby enhancing the overall applicability and efficiency of automated pentesting in real-world scenarios.

Existing Tools. While automated tools exist for individual tasks within these stages, integrating them into a seamless and effective workflow remains a significant challenge. Existing tools specialize in specific penetration testing tasks. For instance, Nmap[29] is widely used for intelligence gathering, allowing testers to analyze network configurations through direct interaction with targets. Nessus[44] and OpenVAS[16] focus on vulnerability analysis, scanning systems for known weaknesses using extensive vulnerability databases. Metasploit[35] is commonly used for exploitation, providing a range of exploits and payloads to execute attacks on identified vulnerabilities. Although these tools are effective alone, their effective use requires expert knowledge, manual decision making, and significant effort to coordinate workflow.

AI-Driven Penetration Testing. Recent advancements in artificial intelligence have led to the development of more sophisticated penetration testing frameworks based on machine learning and Markov Decision Process (MDP) algorithms [6, 19, 49]. For example, Chen et al. [6] designed a reinforcement learning-based framework for automated attack planning. This framework incorporates expert knowledge into state-action pairs and employs a reward function to train the system to execute actions with the highest success rate. Although these frameworks can generate reasonable attack plans, they lack the dynamic implementation aspects of penetration testing. They are unable to react to potential failures and adjust the plan in real time.

LLM-Based Penetration Testing. The rise of LLM-based applications has further advanced the automation of penetration testing tasks such as text analysis, task planning, code modification, and execution debugging. However, the existing LLM-based penetration testing frameworks still lack comprehensive coverage of the stages and automation for practical use. AUTOATTACKER [48] focuses on constructing post-breach attacks, neglecting the pre-compromise stages. PENTESTGPT [12], while implicitly considering multiple stages through its “pentesting task tree,” still relies on human decision to proceed with a certain branch of tasks, leading to inefficiency and ineffectiveness. For example, PENTESTGPT may overly focus on one task while neglecting others, resulting in an unbalanced approach. Moreover, PENTESTGPT and AUTOATTACKER depend on the LLM’s pre-trained knowledge and human analysis to gather additional information about the target, discover and validate vulnerabilities, and select the next steps from the task tree. These tasks still demand considerable manual effort.

Our objective is to develop a comprehensive and automated penetration testing framework that integrates all stages into a coherent and effective workflow. By leveraging the advanced capabilities of

LLMs, we aim to significantly enhance the level of automation in penetration testing, reducing dependency on human expertise and effort while ensuring seamless integration across all stages of the penetration testing process.

2.2 Challenges of Applying LLM to Pentesting

Although LLM-based systems have demonstrated excellent capabilities in various tasks, we identify the following challenges in using LLM in penetration testing tasks.

C1. Limited Pentesting Knowledge. The LLM has basic knowledge about vulnerabilities and penetration testing to help get the penetration testing started. However, it requires the user to manually search for additional information, such as actual CVE numbers, analyze related information about the CVE like the vulnerability type and relevant exploits, manually set up the exploitation tools, search for the appropriate exploit to run, and configure the exploit. All these additional efforts require penetration testing expertise and significant human effort. For example, here is an example conversation with GPT-4 on how to conduct penetration testing on ActiveMQ 5.17.3.

Limited Pentesting Knowledge Example

User: What may be the vulnerabilities of ActiveMQ 5.17.3? And how can I test if the vulnerabilities exist?

GPT-4:

- (1) Keep Software Updated ...
- (2) Check CVEs ...
- (3) Security Scanning Tools ...
- (4) Configuration Review ...
- (5) Penetration Testing ...
- (6) Log Analysis ...

User: Could you give detailed steps for pentesting?

GPT-4:

- (1) Identify Known CVEs ...
- (2) Set Up Metasploit ...
- (3) Search for ActiveMQ Modules ...
- (4) Configure and Run Exploit ...
- (5) Manual Testing ...

C2. Short-term Memory. The limitation of models’ context windows, leading to the short-term memory problem, becomes particularly challenging during long-lasting tasks such as penetration testing, which requires continuous memory across a prolonged time period. For instance, in vulnerability analysis, information gathered during intelligence gathering is crucial for identifying vulnerabilities and searching for corresponding exploits. Similarly, in the exploitation stage, information from the intelligence gathering stage aids in selecting and configuring appropriate exploits. The short-term memory limitations can lead to several issues throughout the penetration testing process.

1) Repetition of Tasks: Due to the restricted context window, the model may forget previously gathered information or actions taken, leading to redundant tasks being performed. For example,

LLM may repeat the information collection process that was already performed earlier.

Repetition of Tasks Example

Intelligence Gathering

LLM: Use Nmap to perform a comprehensive scan of all ports on the target host to identify open ports and services.

User: {Nmap scan results}

Vulnerability Analysis

LLM: Use Nmap to perform a comprehensive scan of all ports on the target host to identify open ports and services.

2) Loss of Context: As the model's context shifts with each interaction or stage transition, it may lose the contextual understanding necessary for making informed decisions or executing sequential tasks effectively. This can result in suboptimal exploitation attempts or misalignment with the overall penetration testing objectives. For example, LLM may fail to provide detailed instructions on how to execute an exploit due to context loss.

Loss of Context Example

Intelligence Gathering

{Information collection steps}...

LLM: The target OS is Linux and the target IP is 192.168.238.129.

Exploitation

User: How do I execute this exploit?

LLM: The target OS and IP are needed to configure the exploit. For investigation of the unknown OS and IP, do the following: ...

C3. Workflow integration. In the context of penetration testing, which involves a multi-stage pipeline of interconnected tasks, integrating an LLM introduces several challenges related to output quality control and stateful working memory management.

1) Output Quality Control: Ensuring that the LLM's output is formatted in a way that downstream modules can parse easily is crucial for the smooth operation of the entire penetration testing pipeline. This requires the LLM to generate output in a structured format that adheres to predefined standards or protocols, making it easier for subsequent modules to process and utilize the information effectively. Additionally, maintaining high content quality is essential. Before passing its output to downstream modules, the LLM should conduct validation checks to ensure the accuracy, completeness, and relevance of the generated information. LLMs may suffer from the hallucination problem, producing irrelevant or incorrect answers. Implementing robust quality control is necessary to mitigate the risk of propagating errors or misleading data through the pipeline, thereby reducing the likelihood of a single point failure disrupting the entire testing process.

2) Stateful Working Memory Management: Each stage of penetration testing often requires different sets of stateful working memory, encompassing information such as discovered vulnerabilities, selected exploits, target environment details, and ongoing session contexts. The challenge lies in enabling smooth transitions

of this working memory between tasks and sessions. If the LLM cannot retain and switch between continuous stateful memory throughout the penetration testing process, it can disrupt the flow and coherence of the testing sequence. For example, if the LLM fails to retain the progress made in exploit execution after obtaining necessary information from the target environment details working memory to proceed, it may lead to restarting the exploit execution from the beginning. This redundancy can delay progress and impact the overall thoroughness and effectiveness of the testing. However, current LLMs do not inherently support such working memory management within and between sessions, posing a significant challenge in achieving seamless integration across the penetration testing pipeline.

2.3 LLM Techniques for Overcoming Challenges

The rapid advancement of LLM studies has introduced a new level of intelligence and automation capabilities, significantly enhancing penetration testing performance. Various LLM techniques can be applied to different stages of pentesting to improve efficiency and effectiveness, addressing the challenges mentioned in §2.2.

LLM agents, which are LLMs equipped with additional tools, extend the functionalities of traditional models. These agents can be beneficial in all stages of pentesting by performing tasks that traditionally required human intervention, such as text analysis and code debugging. With the right tools, an LLM agent can search for and learn penetration testing knowledge online, thus addressing the challenge of limited pentesting knowledge (C1). To fully leverage an LLM agent's capabilities, it is essential to provide an appropriate system message that defines the agent's basic profile, including its capabilities, limitations, output format, and additional specifications [27].

Retrieval-augmented generation (RAG) enhances LLMs by allowing them to utilize external data for generating responses. This technique involves three main stages: indexing, retrieval, and response synthesis. Initially, the dataset is indexed for efficient retrieval. Upon receiving a query, RAG retrieves relevant information from the indexed dataset and combines it with the original query before sending it to the LLM for response synthesis. RAG effectively addresses the challenges of short-term memory (C2) and stateful working memory management (C3.2) by enabling users to maintain long-term memories that can be dynamically queried and stored.

The chain-of-thought (CoT) technique significantly improves the ability of large language models to perform complex reasoning [47]. By guiding the LLM to follow a logical sequence of steps, this method enhances the model's problem-solving capabilities.

Role-playing [22] ask the LLM to impersonate an imaginary character, allowing LLM to operate with clear objectives and boundaries, thereby enhancing their efficiency and effectiveness.

Self-reflection techniques, where the LLM summarizes its past mistakes into long-term memory to avoid similar errors in subsequent communications, have proven useful for learning complex tasks over a handful of trials [41].

Structured output techniques can save time spent on iterative prompt testing and ad-hoc parsing, reducing overall LLM inference costs and latency, as well as developers' effort. Additionally,

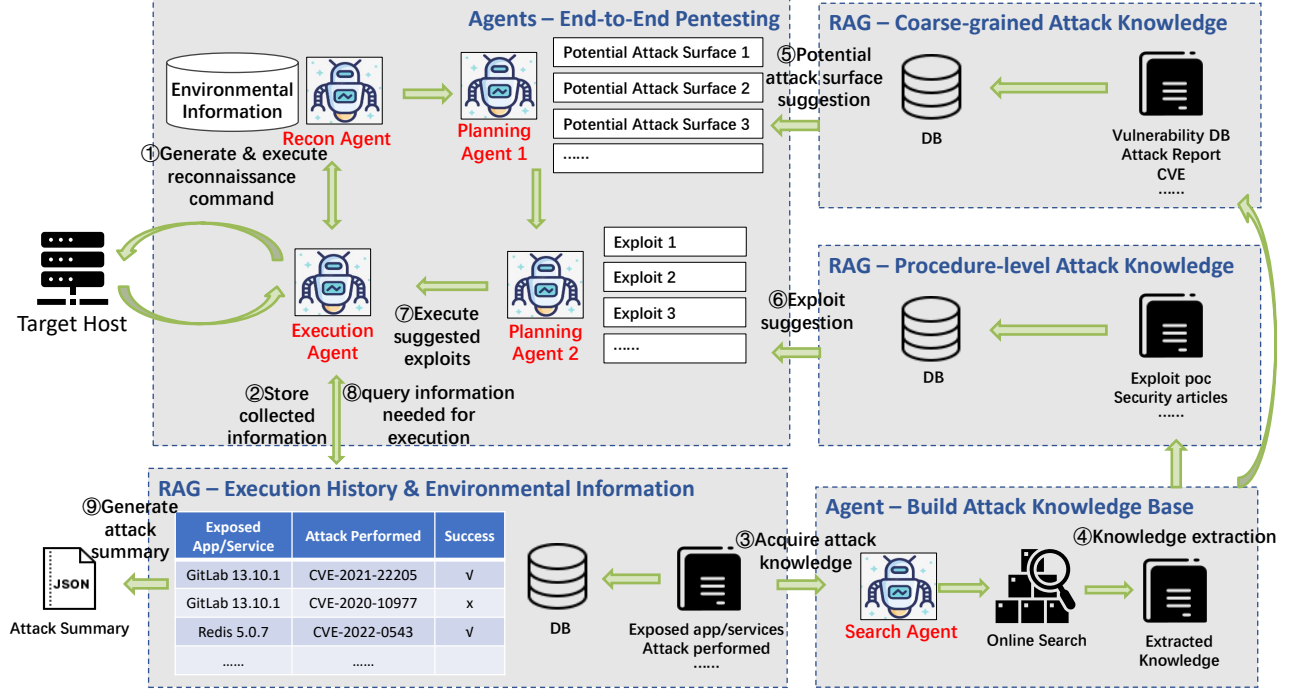


Figure 1: An overview of the components in PENTESTAGENT

structured outputs ensure smooth integration with downstream processes and workflows [24].

Together, chain-of-thought, role-playing, self-reflection, and structured output techniques significantly improve the quality of LLM output, effectively addressing the output quality control challenge (C3.1).

3 System Design

3.1 System Overview

As shown in Fig. 1, PENTESTAGENT comprises four major components: the **reconnaissance agent**, the **search agent**, the **planning agent**, and the **execution agent**. These agents collaborate to perform the three main stages of penetration testing.

Intelligence Gathering: ① Upon receiving user input specifying the target, the reconnaissance agent initiates the penetration testing process by gathering environmental information about the target host. The reconnaissance agent generates and executes reconnaissance commands, aiming to collect comprehensive environmental data from the target host. ② The reconnaissance agent then analyzes the execution results and compiles a summary of the target environment, which is stored in a designated environmental information database.

Vulnerability Analysis: Next, the search and planning agents work together to perform the vulnerability analysis. ③ The search agent queries the environmental information database to retrieve a list of services and applications exposed on the target host. ④ Guided by these services and applications, the search agent searches for potential attack surfaces and procedures and saves them in separate databases. ⑤ The planning agent first leverages the RAG techniques to find a list of potential attack surfaces. ⑥ Subsequently,

the planning agent uses these identified attack surfaces to determine suitable exploits for the target environment.

Exploitation: ⑦ Finally, the execution agent attempts to execute these attack plans on the target host. ⑧ The execution agent communicates with the environmental information database to obtain the necessary information for executing the exploits. It also debugs any execution errors by modifying the code or executing additional commands to gather more information. ⑨ All execution history is stored in a database and can be used to generate a comprehensive penetration testing report.

This structured and automated framework aims to streamline the penetration testing process, enhancing efficiency and reducing the manual effort required.

3.2 Reconnaissance Agent

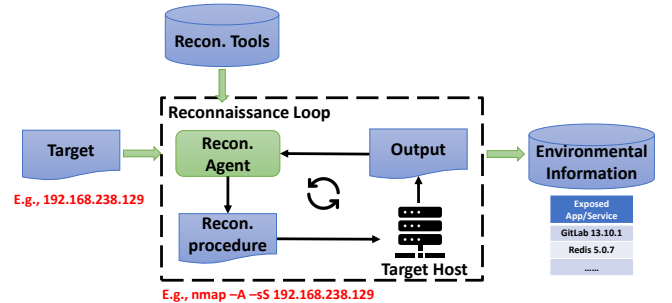


Figure 2: Reconnaissance agent workflow

The reconnaissance agent takes a specified target as input and interacts with it to collect detailed information, ultimately generating a summary of the environmental information as the output. As